P2664R5
Extend std::simd with permutation API

Published Proposal, 2023-10-25

This version:
http://wg21.link/P2664R5

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to extend `std::simd` with features enabling permutation of data within and across SIMD values.

Table of Contents

§ 1. Motivation

ISO/IEC 19570:2018 introduced data-parallel types to the C++ Extensions for Parallelism TS. [P1915R0] asked for feedback on `std::simd` in Parallelism TS 2. [P1928R4] aims to make that a part of C++ IS. Intel supports the concept of a standard interface to SIMD instruction capabilities and has made further suggestions for other APIs and facilities for `std::simd` in document [P2638R0].

One of the extra features we suggested was the ability to be able to permute elements across or within SIMD values. These operations are critical for supporting formatting of data in preparation for other operations (e.g., transpose, strided operations, interleaving, and many more), but are relatively poorly supported in the current `std::simd` proposal. Many modern SIMD processors include sophisticated support for permutation instructions so in this document we shall propose a set of API functions which make those underlying instructions accessible in a generic way.

In this document we shall start with some background into the current `std::simd` proposal and extensions, describe the state of the art on other simd libraries, and then describe a proposal for some new API functions to add to `std::simd`.

2. Revision History

R4 => R5

- Renamed `gather` and `scatter` to `gather_from` and `scatter_to`.
- Made masking overloads for `gather_from` and `scatter_to` part of the proposal rather than a design option.
- Added Flags to `gather_from` and `scatter_to` to control memory behaviour.
- Added a design option discussing the merits of making `gather_from` and `scatter_to` member functions.
- Switched to using new names for functions like `simd_split` and `simd_cat`.

R3 => R4

- Renamed `simd`/`simd_mask` to `basic_simd`/`basic_simd_mask` to match [P1928R4].
- Replaced deduced masks with nested typedefs (i.e., use `basic_simd<T, ABI>::mask_type` instead of `basic_simd_mask<T, MaskABI>`).
- Replaced fixed_size types with their new counterparts (e.g., `fixed_size_simd<T, 5>` would now be `simd<T, 5>`).
- Removed design option to allow `compress` and `expand` to take simd value parameters which are different sizes. This would always lead to some silent loss of data (either the mask or the values being compressed).
- Reordered mask parameter position (it now matches `simd_select`, `copy_to`, etc., by having the mask appear first).

R2 => R3

- Added more detail about compiler's ability to optimise a generated sequence
- Made memory operations (gather/scatter) a class of its own, rather than trying to make them appear like special forms of the other types of permute.
- Moved special indexes and generator size parameter description into main body of text rather than being a design option.
- Added fill value parameters for compress and expand functions.
- Improved wording and examples for memory permutations (gather/scatter).
- Added wording for static permute, dynamic permute, mask permute and gather/scatter.

R1 => R2

- Rewrite to bikeshed
- Add more content for proposal overview.

R0 => R1

- Add polls from Kona SG1 2022

§ 3. Background

In ISO/IEC 19570:2018 there were only a few functions which could be used to achieve any sort of element permutation across or within `simd<>` values. Those functions were relatively modest in scope and they built upon the ability to split and concatenate simd values at compile-time. For example, a simd value containing 5 elements could be split into two smaller pieces of sizes 2 and 3 respectively:

```
simd<float, 5> x;
…
const auto [piece0, piece1] = simd_split<3>(x);
```

Those smaller pieces could be acted on using any of the `std::simd` features, and then later recombined using `simd_cat`, possibly in a different order or with duplication:

```
const auto output = simd_cat(piece1, piece1, piece0);
```

The `simd_split` and `simd_cat` functions can be used to achieve several permutation effects, but they are unwieldy for anything complicated, and since they are also compile-time they preclude any dynamic runtime permutation.

In [P2638R0] Intel suggested extra operations to insert and extract SIMD values to and from other SIMD containers:

```
const auto t = extract<2, 5>(v); // Read out 3 SIMD values, starting from p
insert<5>(v, t); // Put the 3 previously read values back into the containe
```

These are convenient, but they still don't allow arbitrary or expressive permute operations.

Two suggestions were made in [P0918R1]: add an arbitrary compile-time shuffle function, and add a specific type of permutation called an interleave. The compile-time shuffle can be used as follows:

```
const auto p = shuffle<3, 4, 1, 1, 2, 3>(x);
// p::value_type will be the same as x::value_type
// p::size will be 6
```

The indexes can be arbitrary, and therefore allow any duplication and reordering of elements. The output `std::simd` object will have the same number of elements as there are supplied indexes, and the type of the SIMD output will be that of the input. It is noted also in the shuffle API that this function can be applied to `basic_simd_mask` values too, and that shuffling across multiple SIMD values can be achieved by first concatenating the values into a single larger SIMD:

```
const auto p = shuffle<10, 9, 0, 1>(simd_cat(x, y));
```

In addition to the shuffle function, [P0918R1] also proposes a function called interleave which accepts two SIMD values and interleaves respective elements into a new SIMD value which is twice the size. For example, given inputs `[a0 a1 a2]` and `[b0 b1 b2]`, the output would be `[a0 b0 a1 b1 a2 b2]`. Interleaving is a common operation and is often directly supported by processors with explicit instructions (e.g., Intel's `punpcklwd`), so as in [P0918R1] and also in other simd libraries there will be named functions which expose that instruction. There will be other common permutation operations which also have specialist hardware support and it is common for them to be exposed in named functions also. For example, [Highway] provides `DupOdd`, `DupEven`, `ReverseBlocks`, `Shuffle0231`, and so on, which map efficiently to underlying instructions. While this can ensure that good code is generated for specific function, it does mean that:

- the set of named functions can only include those features available on all potential targets
- portability is reduced by exposing functions only on those targets which support them.

Neither of these is desirable, and in our suggestions below we provide an alternative.

§ 4. Extending `std::simd` to support permutations

There are four classes of permutation which could be supported by `std::simd`;

- using compile-time computed indexes
- using another SIMD as a dynamic index
- using a bit-mask for selecting active elements
- permuting memory (gather and scatter)

Modern processor instruction sets typically support all 4 of these classes of instruction. In the following sections we shall examine in more detail each of these classes and the potential APIs needed to expose those features to the user. We shall also examine what it means to permute memory, and whether named functions should be provided for common permutation patterns.

`std::simd` could be modified to allow permutations either by adding to the base definition of `std::simd` itself (specifically, to include a new `simd::operator[]` or `simd_mask::operator[]`), or by introducing overloaded free functions which extend `std::simd`. We shall consider both options below.

Note that a `basic_simd_mask` is a special case of a type of SIMD and can therefore be permuted in the same way as a conventional SIMD. In our discussions below we assume that the permutations would apply equally to a `basic_simd_mask` unless we note otherwise.

§ 4.1. Using compile-time computed indexes

The first proposal is to provide a `permute` function which accepts a compile-time function which is used to generate the indexes to use in the permutation. This is a very powerful concept: firstly, it allows the task of computing the indexes to be offloaded to the compiler using an index-generating function, and secondly it works on `simd<>` values of arbitrary size. It's definition would be something like:

```
template<std::size_t SizeSelector = 0, typename T, typename Abi, typename F
constexpr simd<T, output-size>
permute(const basic_simd<T, Abi>& v, PermuteGenerator&& fn)
```

It can be used as in the following example:

```
const auto dupEvenIndex = [](size_t idx) -> size_t { return (idx / 2) * 2;
const auto de = permute(values, dupEvenIndex);
```

Note that the permutation function maps from the index of each output element to the index which should be used as the source for that position. So, for example, the `dupEvenIndex` function would map the output indexes `[0, 1, 2..)` to the source indexes `[0 0 2 2 4 4…)`. This example permutation is common enough that it has hardware support in Intel processors, and the compiler can map the above function directly to the corresponding instruction:

```
vmovsldup ymm0, ymm0
```

By default, the permute function will generate as many elements in the output simd as there were input values, but the output size can be explicitly specified if it cannot be inferred or it needs to be modified. For example, the following permute generates 4 values with a stride of 3 (i.e., indexes `[0, 3, 6, 9]`).

```
const auto stride3 = permute<4>(values, [](size_t idx) -> size_t { return idx * 3; });
```

§ 4.1.1. Special generator indexes

The obvious index representation to return from each generator invocation is to return an integral value in the range `[0..size)`. The compiler can enforce at compile-time that the index is in the valid range and dynamic indexes would not be permitted. In addition, there are three other types of index value which can be returned which give the compiler even more flexibility.

- Negative indexes represent reverse indexes, which are intepreted as indexes taken from the end of the simd instead of the beginning. So -1 would be the last element, -2 the penultimate element, and so on.
- The special index named `simd_zero_element` will instead a zero value at the indicated element position, rather than copying a value from the input simd.
- The special index named `simd_uninit_element` will indicate to the compiler that the element at the indicated position can be left in an uninitialised state. This gives an opportunity for more efficient code generation where the programmer knows that an element is unused.

Here is an example where zeros are inserted into the even-indexed output positions of a SIMD value while the odd-indexed values are copied directly from the input.

```
permute(v, [](auto idx) { (idx % 2) ? simd_zero_element : idx; });
```

4.1.2. Generator function size argument

It can be useful to create libraries of common permutation generator functions to handle common use cases, but such functions need to be able to know the extent of the indexes that they are allowed to permute. The generator function is allowed to accept an optional `std::size_t` argument giving the size of the SIMD value being permuted. For example, the following generator extracts elements of a SIMD starting at the half-way point:

```
auto topHalf = [](std::integral auto idx, std::size_t simd_size) {
  return (simd_size / 2) + idx;
};
```

4.1.3. Design option - more special constants

There are other possible special constants that could be given special status in the compile-time permute, including the ability to insert NaN, +ve infinity, -ve infinite, max_value, min_value, and so on. Should these be adopted too?

4.1.4. Implementation Experience

In other simd or vector libraries which provide named specific permute functions, the reason often given is that it allows specific hardware instructions to be used to efficiently implement those permutes. For example, Intel has the `vmovsldup` instruction as noted above, so some libraries provide functions in their API with names which reflect this (e.g., such a function might be called `DupLow`). The disadvantages are that it hinders portability, and only allows access to the hardware features that the authors of the library have chosen to expose.

Using the generator-based approach makes it easier to access a wide range of underlying special purpose instructions, or to fall back to equivalent instructions sequences where they are not available, but it is desirable that such behavior should be efficient. To judge whether the generator-based permutation API was efficient and useful we implemented the extensions in the Intel `std::simd` library and looked at the performance of different compile-time patterns and their generated

instruction sequences. We found that GCC and LLVM were able to determine the correct hardware instructions to use for a variety of common permutation patterns, and in some cases the compiler was even able to use the side-effects of other instructions to effect permutations in ways that human programmers had overlooked. In all cases, where the pattern was too complicated to map to native hardware features the compiler fell back to using general purpose permutation patterns, such as Intel's `permutex2var` family of instructions.

The generator function works by creating a list of compile-time constants which are used to perform the permutation. Because the list is compile-time it can be generated using completely arbitrary (and potentially very inefficient) code, but the outcome will always be a list of constants. Therefore the compiler's ability to generate a permutation is dependent upon how well it can convert a list of constants into a permutation, not how well the programmer is at writing clear permute functions.

As a small illustration of the compiler's ability, the following table shows some compile-time function permute calls and the code that the LLVM 14.0.0 compiler has generated for each:

Permute call	Purpose	Output from clang 14.0.0
`permute (x, [](auto idx) { return idx & ~1; });`	Duplicate even elements	`vmovsldup zmm0, zmm0`
`permute (x, [](auto idx) { return idx ^ 1; });`	Swap even/odd elements in each pair (complex-valued IQ swap)	`vpermilps ymm0, ymm0, 177`
`permute<8>(x, [](auto idx) { return idx + 8; });`	Extract upper half of a 16-element vector. Note that the instruction sequence accepts a zmm input and returns a ymm output.	`vextractf64x4 ymm0, zmm0, 1`

In each case the compiler has accepted the compile-time index constants and converted them into an instruction which efficiently implements the desired permutation. We can safely leave the compiler to determine the best instruction from an arbitrary compile-time function.

§ 4.2. Using another SIMD as the dynamic index

The second proposal for the permute API is to allow the required indexes to be passed in using a second SIMD value containing the run-time indexes:

```
template<typename T, typename AbiT, std::integral Idx, typename AbiIdx>
constexpr simd<T, basic_simd<Idx, AbiIdx>::size()>
permute(const basic_simd<T, AbiT>& v, const basic_simd<Idx, AbiIdx>& indexe
```

This can be used as in the following example:

```
simd<unsigned, 8> indexes = getIndexes();
simd<float, 5> values = getValues();
...
const auto result = permute(values, indexes);
// result::value_type is float
// result::size is 8
```

The permute function will return a new SIMD value which has the same element type as the input value being permuted, and the same number of elements as the indexing SIMD (i.e., float and 8 in the example above). The permute may duplicate or arbitrarily reorder the values. The index values must be of integral type, with no implicit conversions from other types permitted. The behavior is undefined if any index is outside the valid index range. Dynamic permutes across multiple input values are handled by concatenating the input values together. The indexes in the selector will only be treated as indexes, and will never have special properties as described above (e.g., no zero element selection)

In addition to the function called permute, a subscript operator will also be provided:

```
template <std::integral Idx, typename AbiIdx>
constexpr simd<T, basic_simd<Idx, AbiIdx>::size()>
simd::operator[](const basic_simd<Idx, AbiIdx> &indexes) const;
```

Here is an example of how this could be used:

```
simd<int, 8> indexes = getIndexes();
simd<float, 5> values = getValues();
...
```

```
const auto result = values[indexes];
// result::value_type is float
// result::size is 8
```

4.2.1. Design option - C++23 multi-index subscript

Sometimes the subscripts to use in a permute might be known at compile time, but to use them in a permutation requires them to be put into an index simd first:

```
constexpr simd<int> indexes = {3, 6, 1, 6}; // Note – assumes initialiser l
auto output = values[indexes];
```

In C++23 multi-subscript operator was introduced and this could be used to allow a more compact representation:

```
auto output = values[3, 6, 1, 6];
// output::value_type is values::value_type
// output::size is 4
```

It would be good if it could also appear on the left-hand-side;

```
output_simd[2, 5, 6] = x; // Overwrite selected elements with a single valu
```

Should non-constant indexes be allowed too? Our experience with GCC and LLVM suggests that work would be needed on their code generation for non-constant variable indexes to be improved though, as they are both currently poor at this.

Should `operator[]` be reserved for slicing or sub-ranges?

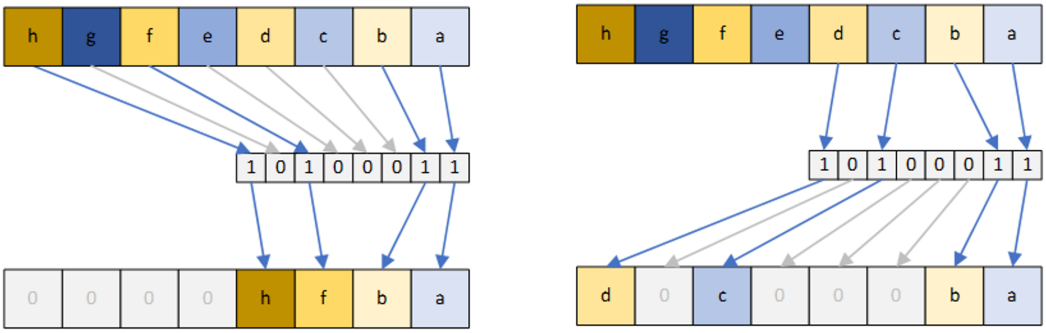
4.2.2. Implementation experience

We implemented the runtime-indexed permute API in Intel's example `std::simd` implementation. Small index operations (e.g., indexing one native sized simd by another) were mapped directly onto the underlying native instruction and so were as efficient as calling an intrinsic. More complicated

cases for permutation, such as where either the index or data SIMD parameters were bigger than a native register are also handled with comparable efficiency to hand-written intrinsic code.

§ 4.3. Permutation using simd masks (compress/expand)

A third variant of permutation is where a `basic_simd_mask` is used as a proxy for an index sequence instead. The presence or absence of an element in the `basic_simd_mask` will cause the element to be used or not. The following diagrams illustrate this:



On the left the values in the SIMD are compressed; only those elements which have their respective `basic_simd_mask` element set will be copied into the next available space in the output SIMD. Their relative order will be maintained, and the selected elements will be stored contiguously in the output. The output value will have the same number of elements as the input value, with any unused elements being left in a valid but unspecified state. The behavior of the function is similar to `std::copy_if`, although other simd libraries may call this function compression or compaction instead. The expansion function on the right of the diagram has the opposite effect, copying values from a contiguous region to potentially non-contiguous positions indicated by a mask bit. The two functions have prototypes as follows:

```
template<typename T, typename Abi>
constexpr basic_simd<T, Abi>
compress(const typename basic_simd<T, Abi>::mask_type& mask, const basic_si

template<typename T, typename Abi>
constexpr basic_simd<T, Abi>
compress(const typename basic_simd<T, Abi>::mask_type& mask,
         const basic_simd<T, Abi>& value, T fill_value);

template<typename T, typename Abi>
```

```
constexpr basic_simd<T, Abi>
expand(const typename basic_simd<T, Abi>::mask_type& mask,
       const basic_simd<T, Abi>& value, const simd<T, Abi>& original);
```

In both functions all `basic_simd` or `basic_simd_mask` values must have the same `value_type` and `size`.

A compression operation is typically used to throw away unwanted values (i.e., values which do not satisfy the mask condition). Unwanted positions in the output will be left unspecified, unless the user supplies a third parameter specifying the value which should be used to fill those unwanted positions.

An expansion operation is typically used to overwrite selected elements of some existing SIMD value with new values taken from the contiguous beginning of another SIMD value. The expansion operation therefore has a slightly different meaning for its fill value.

§ 4.3.1. Unused value element initialisation

In the design shown above the `compress` function comes in two variants, the first of which leaves unused values in the output in an unitialised but valid state, and the second function provides a default value to use for the initialisation. Is is worth looking at this in more detail to understand why there isn't a single function which always initializes its unused elements. Consider a compression operation:

```
simd<int, 6> values = ...;
simd_mask<int, 6> mask = ...;

auto compressed = compress(values, mask);
// compressed::value_type is int
// compressed::size is 6
```

The output value has the same size as the mask selector, but the actual mask bits won't be known until run-time. This raises the question of what values should be put into the unused output elements.

The current proposal would leave the unused elements in a valid but unspecified state. This is comparable to functions like `std::shift_left` and `std::shift_right`. The alternative is for the compress function to leave the unused values in a value-initialized state.

The advantage of an unspecified state is that the code doesn't need to make a special effort to insert values which might not be used anyway, but has the disadvantage that the unspecified state might

catch out the unwary user. Using a value initialized state helps the unwary user by providing a default state which is sane and repeatable, but may come with a performance cost.

Intel have an example implementation of `std::simd`, and our experience with that demonstrates that when native support is available (e.g., with AVX-512 ISA) it makes no difference to performance whether the unused values are initialising to specific value or left uninitialized, since the instruction itself inserts the values into unused positions as an integrated part of its operation. However, a synthesized code sequence is more expensive when setting the elements to a specific value since it is necessary to also compute the population count of the mask, turn that into a mask and then arrange for the unused values to be overwritten with something else using that mask.

Given that the compress function is essentially only extracting valid values from the input SIMD and discarding the others, and that there is a potential performance penalty that comes from initialising unused values, then we think that the default should be to leave unused elements in unspecified states.

Related to this is what to do when a programmer does want to initialize unused values to a known value. With the proposed interface the programmer would be required to compute a new partial mask from their original selection mask (i.e., compute the population count of the original mask, and then turn that into a mask of the lower elements), and then use that to conditionally blend in their desired value. This can be inefficient, and for targets like Intel AVX-512, it doesn't exploit the hardware's ability to automatically insert a value anyway. For this reason we propose that an overload of `compress` is provided that accepts a third parameter giving a value to insert into unused elements:

```
template<typename T, typename Abi>
basic_simd<T, Abi>
compress(const basic_simd<T, Abi>& value,
         const typename basic_simd<T, Abi>::mask_type& mask,
         T default_value);
```

This makes the programmer's intent explicit in the code, it simplifies the code, and it allows those targets which support automatic insertion of a value to efficiently make use of that feature.

§ 4.3.2. Implementation Experience

In Intel's example implementation of `std::simd` we have implemented permute-by-mask. On modern processors which support AVX-512 or VBMI2 we found the mapping from `compress` or `operator[]` to be efficient, and allowed a natural representation of this compaction operation where needed. Implementing permute-by-mask on older processors which lacked native support was a little harder, but could still be implemented as efficiently as an experienced programmer could achieve

using hand-written intrinsics. Determining the most efficient implementation under all conditions however showed that less experienced programmers would struggle to implement this feature. Therefore, making this feature available in `std::simd` itself removed any non-portable calls to native compression/expansion instructions, and also allowed the library implementer to use efficient sequences for different scenarios.

§ 4.4. Memory-based permutes

When permuting values which are stored in memory, the operations are normally called gather (reading data from memory), or scatter (writing values to memory). We propose that four functions are provided which implement these operations, with and without masking. Firstly, the gather function:

```
template<std::contiguous_iterator Iter, std::integral Idx, typename AbiIdx,
constexpr simd<std::iter_value_t<Iter>, basic_simd<Idx, AbiIdx>::size()>
gather_from(Iter in, const simd<Idx, AbiIdx>& indexes, simd_flags<Flags...>

template<std::contiguous_iterator Iter, std::integral Idx, typename AbiIdx,
constexpr simd<std::iter_value_t<Iter>, basic_simd<Idx, AbiIdx>::size()>
gather_from(Iter in, const typename simd<Idx, AbiIdx>::mask_type& mask,
            const simd<Idx, AbiIdx>& indexes, simd_flags<Flags...> f = {});
```

The gather operation returns a `simd` which has elements of the iterator type (i.e., no conversion is possible from one iterator to a different simd element type) with one simd element for every supplied index.

In the masked overload the mask's parameter type matches that of the supplied indexes since it is effectively turning indexes on and off. Any values which are unmasked will be value initialised in the result.

Both overloads accept a default flags parameter which can control behaviour such as element alignment and streaming operations (i.e., cache bypass). These flags allow the same control over the memory operations as with `copy_from`, `copy_to`, and the memory-loading constructors.

The scatter function is defined as:

```
template<std::contiguous_iterator Iter,
         typename T, typename TAbi,
         std::integral Idx, typename IdxAbi, Flags...>
constexpr void
```

```
scatter_to(const simd<T, TAbi>& values, Iter out,
           const simd<Idx, IdxAbi>& indexes, simd_flags<Flags...> f = {});

template<std::contiguous_iterator Iter,
         typename T, typename TAbi,
         std::integral Idx, typename IdxAbi, Flags...>
constexpr void
scatter_to(const simd<T, TAbi>& values, Iter out,
           const typename simd<Idx, AbiIdx>::mask_type& mask,
           const simd<Idx, IdxAbi>& indexes,
           simd_flags<Flags...> f = {});
```

In the masked overload, the mask parameter type has been chosen to match the index type, which is consistent with the `gather_from` function. The values written to memory will be of the iterator type, with a conversion from the input type taking place if they are different. As a memory operation, `scatter_to` also accepts a `simd_flags` parameter.

Note that for both `gather_from` and `scatter_to` the names and argument orders have been chosen to mirror `copy_to` and `copy_from` to make their relationship apparent.

The `gather_from` function could be used as follows:

```
int array[1024];
...
const auto r1 = gather_from(array, indexes);
```

The scatter could be used as follows:

```
int array[1024];
simd<int> values = ...;
simd<int> indexes = ...;
simd<int>::mask_type mask = ...;
...
scatter_to(values, array, mask, indexes);
```

4.4.1. Design option - allow subscripting a contiguous iterator

If [DNMSO] is allowed, then a pointer or `std::contiguous_iterator` could be used directly for unmasked gathers and scatters:

```
m = v.begin();
auto data = m[simd_values];

m[simd_values] = inputs; // Scatter to memory
```

4.4.2. Design option - make `gather_from` and `scatter_to` member functions

At Varna 2023 there was the suggestion to make `gather_from` and `scatter_to` become member functions. Reasons for this included:

- to match `copy_from` and `copy_to`, which are the closest equivalents to these memory operations that already exist, and which are already members (though they are the only ones).
- to allow `gather_from` to perform a conversion from the iterator type to the object's own element type. Note that `scatter_to` already accepts an iterator of one type and a value simd of another element type, so scatter-conversion is already supported.
- to allow a masked `gather_from` to partially overwrite an existing simd value (i.e., masked `gather_from` currently has no way of specifying what value should be used for unused elements and uses value initialization).

After further thought and discussion since the Varna meeting, we have the following observations.

The alternative to having `gather_from` be able to perform conversion is to explicitly convert after the gather has completed. For example:

```
int16_t* ptr;

// Load as int16_t
auto g = gather_from(ptr, indexes);

// Convert to a different type
auto gf = simd<float>(g);
```

It is likely that the compiler will efficiently merge the code for the gather and the conversion anyway, so the only advantage of having a gather as a member function would be conciseness and uniformity. Note again that `scatter_to` already allows conversion anyway, so it makes no difference whether `scatter_to` is a member or not for this purpose.

If `gather_from` was added, then it should probably also exist as a constructor variant too, like its `copy_from` counterpart. For example, it is common to use gather to read values from memory into a new object. If `gather_from` was a member, then the call would look like this:

```
auto g = simd<float>().gather_from(ptr, indexes);
```

To avoid this slightly clunky syntax, `simd` should provide a constructor allowing the same parameters to be used directly (which is also the case with `copy_from`):

```
auto g = simd<float>(ptr, indexes);
```

This does lose something in readability though, as the call site doesn't immediately suggest a gather operation is taking place. This could be overcome by adding a "named constructor":

```
auto g = simd<float>::gather_constructor(ptr, idx)
```

But then there are currently no other named constructors, so what is special about `gather_from` to deserve this additional name? this opens up a discussion which is outside the scope of this paper.

This last point can be extended to observe that there are no member functions in `simd` other than `copy_to` and `copy_from`. What is special about those functions that mean that they should differ to all the other free functions (e.g., `simd_cat`, `simd_select`, `simd_split`, `compress`, `expand`, `permute`). Again, this opens up a much wider discussion which is also beyond the scope of this paper

For now, we propose to leave `gather_from` and `scatter_to` as free functions since there is no compelling technical reason for making them members. We can change this in future if separate discussions fall in favour of having named constructors and member functions elsewhere.

§ 5. Named functions

The APIs discussed in this document are very flexible and can be used to build effectively arbitrary permutations. However, there are inevitably certain patterns that are very common and it could be convenient to provide named functions for those patterns. For example:

```
auto dupEven(simdable v) {
    return permute(v, [](size_t idx) -> size_t { return (idx / 2) * 2; });
};
auto dupOdd(simdable v) {
    return permute(v, [](size_t idx) -> size_t { return idx | 1; });
};
auto swapOddEven(simdable auto v) {
    return permute(v, [](size_t idx) -> size_t { return idx ^1; });
}
auto even(simdable auto v) {
    return permute<v::size() / 2>(v, [](size_t idx) -> size_t { return idx
}
```

In many cases there are already names in C++ that reflect the operation. Providing overloads in `std::simd` which create SIMD functions with similar names to their existing C++ counterparts makes it clear to the programmer what the function is doing when applied to a SIMD value. For example:

```
std::rotate
std::shift_left
std::shift_right
std::copy_if
std::remove_if
std::slice // tricky - used for valarray at the moment, but it
           // also captures concepts like odd and even
std::stable_partition // Use a mask to split the values
```

These functions could be added at a later date.

§ 6. Summary

In this document we have described four styles of interface for handling permutes: compile-time generated, simd-indexed, mask-indexed, and memory based. In combination, these interfaces allow all types of common permute to be expressed in a way which is clean, concise, and efficient for the compiler to implement.

§ 7. Wording

§ 7.1. Add new section [**simd.static_permute**]

`simd` static permute [simd.static_permute]

```
constexpr static int simd_zero_element   = /* Implementation defined */;
constexpr static int simd_uninit_element = /* Implementation defined */;
```

Constraints:

- These cannot take values which could be mistaken for valid index values, so they must be outside the range `[0..max-impl-size)`.

Remarks:

- Special constants can be returned by the permutation generator function to indicate special behavior of that element.

```
// Free function to generate compile-time permute of simd
template<std::size_t SizeSelector = 0, typename T, typename Abi, typename
constexpr simd<T, output-size>
permute(const basic_simd<T, Abi>& v, PermuteGenerator&& fn)

// Free function to generate compile-time permute of simd_mask
template<std::size_t SizeSelector = 0, typename T, typename Abi, typename
constexpr simd_mask<T, output-size>
permute(const basic_simd_mask<T, Abi>& v, PermuteGenerator&& fn)
```

Constraints:

- `std::integral<std::invoke_result<PermuteGenerator, std::size_t> || std::integral<std::invoke_result<PermuteGenerator, std::size_t, std::size_t>` is `true`.

Returns:

- A `basic_simd` or `basic_simd_mask` object of size `output-size` (see below) where the i^{th} element is initialized as follows. For each output index `[0..output-size)` the result of the expression `simd(gen(integral_constant<size_t, i>()))`, or `simd(gen(integral_constant<size_t, i>(),`

`integral_constant<size_t,output-size>())` is computed. The result index `x` is used to select a value from `v` as follows:

- A value in the range `[0..size)` will use `v[x]`.
- The value `simd_zero_element` will use `T()`.
- The value `simd_uninit_element` will use a valid but unspecified value of the compiler's choosing.
- Any other value is undefined.

The `output-size` will be the same as `v.size` if `SizeSelector` is zero, otherwise it will be set to `SizeSelector`.

Remarks:

- The calls to the generator are unsequenced with respect to each other.

§ 7.2. Add new section [**simd.dynamic_permute**]

`simd` dynamic permute [simd.dynamic_permute]

```
// Free function to permute simd by dynamic index
template<typename T, typename AbiT, std::integral Idx, typename AbiIdx>
constexpr simd<T, basic_simd<Idx, AbiIdx>::size()>
permute(const basic_simd<T, AbiT>& v, const basic_simd<Idx, AbiIdx>& inde

// Free function to permute simd mask by dynamic index
template<typename T, typename AbiT, std::integral Idx, typename AbiIdx>
constexpr simd_mask<T, basic_simd_mask<Idx, AbiIdx>::size()>
permute(const basic_simd_mask<T, AbiT>& v, const basic_simd_mask<Idx, Abi

// Member function to permute simd by dynamic index using subscript opera
template<std::integral Idx, typename AbiIdx>
constexpr simd<T, basic_simd<Idx, AbiIdx>::size()>
basic_simd::operator[](const basic_simd<Idx, AbiIdx>& indexes)

// Member function to permute a simd mask by dynamic index using subscrip
template<std::integral Idx, typename AbiIdx>
constexpr simd_mask<T, basic_simd<Idx, AbiIdx>::size()>
basic_simd_mask::operator[](const basic_simd<Idx, AbiIdx>& indexes)
```

Preconditions:

- All values in `indexes` must be in the range `[0..size)`.
- All index elements must be integral types.

Returns:

- A `basic_simd` or `basic_simd_mask` object of size `simd_size_v<Idx, AbiIdx>` where the i^{th} element is a copy of `v[indexes[i]]` if i is in the range `[0..v.size)` or undefined otherwise.

§ 7.3. Add new section [**simd.mask_permute**]

`simd` mask permute [simd.mask_permute]

```
// Free function to compress simd values using a mask selector
(1) template<typename T, typename Abi>
constexpr basic_simd<T, Abi>
compress(const basic_simd<T, Abi>::mask_type& selector, const basic_simd<

// Free function to compress simd_mask elements using a mask selector
(2) template<typename T, typename Abi>
constexpr basic_simd_mask<T, Abi>
compress(const basic_simd_mask<T, Abi>& selector,
         const basic_simd_mask<T, Abi>& v)

// Free function to compress simd values using a mask selector
// with a fill value for unused elements
(3) template<typename T, typename Abi>
constexpr basic_simd<T, Abi>
compress(const basic_simd_mask<T, Abi>::mask_type& selector,
         const basic_simd<T, Abi>& v, T fill_value)

// Free function to compress simd_mask elements using a mask selector
// with a fill value for unused elements
(4) template<typename T, typename Abi>
constexpr basic_simd_mask<T, Abi>
compress(const basic_simd_mask<T, Abi>& selector,
         const basic_simd_mask<T, Abi>& v, T fill_value)
```

Returns:

- A `basic_simd` or `basic_simd_mask` object in which the first `[0..reduce_count(selector))` values are copies of those elements in `v` whose corresponding mask element is set. The relative order of the values from `v` is unchanged.
- For (1) and (2) output values in the range `[reduce_count(selector)...v.size))` will be valid but unspecified values of type T.
- For (3) and (4) output values in the range `[reduce_count(selector)...v.size))` will be copies of `fill_value`.

Remarks:

- The `compress` operation is used to throw away unwanted values, so the `fill_value` is used to put sane values into unused positions. This is in contrast to `expand` where the operation is typically used to overwrite selected elements of an existing SIMD value and so unused positions come from another SIMD value.

```
// Free function to expand simd values using a mask selector
template<typename T, typename Abi>
constexpr basic_simd<T, Abi>
expand(const typename basic_simd<T, Abi>::mask_type& selector, const basi
       const basic_simd<T, Abi>& original = ())

// Free function to expand simd_mask elements using a mask selector
template<typename T, typename Abi>
constexpr basic_simd_mask<T, Abi>
expand(const basic_simd<T, Abi>::mask_type& selector, const basic_simd_ma
       const basic_simd<T, Abi>& original = ())
```

Returns:

- A `basic_simd` or `basic_simd_mask` object in which the first `[0..reduce_count(selector))` values are copied to those output elements whose respective selector elements are set. Any output elements whose respective selector elements are not set will have the corresponding element from `original` copied into that position.

Remarks:

- `expand` is often used to overwrite selected positions in an existing SIMD value, so these functions take a complete SIMD value as the source from which to take unused mask selector elements. This is in contrast to `compress` where the compression operation is effectively throwing away unwanted elements and the `fill_value` is being used to put sane values into unused positions.

§ 7.4. Add new section [**simd.memory_permute**]

simd memory permute [simd.memory_permute]

```
template<std::contiguous_iterator Iter, std::integral Idx, typename Abi,
constexpr simd<std::iter_value_t<Iter>, simd<Idx, Abi>::size>
gather_from(Iter in, const simd<Idx, Abi>& indexes, simd_flags<Flags...>

template<std::contiguous_iterator Iter, std::integral Idx, typename Abi,
constexpr simd<std::iter_value_t<Iter>, simd<Idx, Abi>::size>
gather_from(Iter in, const typename simd<Idx, AbiIdx>::mask_type& mask,
       const simd<Idx, Abi>& indexes, simd_flags<Flags...> f = {});
```

Constraints:

- `std::integral<Idx>` is `true`.

Preconditions:

- All values in `indexes` must refer to elements which are within the range of the contiguous input iterator.

Returns:

- A `basic_simd` object with the same number of elements as `indexes` and the type of the element to which `Iter` refers. The i^{th} element of the object will be a copy of the value `in[indexes[i]]`, or if a mask is supplied and its respective bit is false then return a value initialized element.

```
template<typename T, typename TAbi,
         std::contiguous_iterator Iter,
         std::integral Idx, typename IdxAbi, Flags...>
constexpr void
scatter_to(const simd<T, TAbi>& value, Iter out, const simd<Idx,
        IdxAbi>& indexes, simd_flags f = {});

template<typename T, typename TAbi,
         std::contiguous_iterator Iter,
         std::integral Idx, typename IdxAbi, Flags...>
constexpr void
```

```
scatter_to(const simd<T, TAbi>& value, Iter out,
        const typename simd<Idx, AbiIdx>::mask_type& mask,
        const simd<Idx, IdxAbi>& indexes, simd_flags<Flags...> f = {});
```

Constraints:

- `std::integral<Idx>` is `true`.
- `simd_size_v<simd<T, TAbi>> == simd_size_v<simd<Idx, IdxAbi>>` is `true`.
- `std::convertible_to<T, std::iter_value_t<Iter>>` is `true`.

Preconditions:

- All values in `indexes` must refer to elements which are within the range of the contiguous output iterator.

Mandates: If the template parameter pack Flags does not contain the type identifying `simd_flag_convert`, then the conversion from `T` to `iter_value_t<It>` is value-preserving.

Effects:

- For all i in the range `[0..simd_size_v<Idx, IdxAbi>)` the value at `out[indexes[i]]` will be a copy of `value[i]` converted from `T` to the iterator type, unless a mask is provided and `mask[i]` is false in which case no memory write for that element will occur.

Remarks:

- The order in which the individual memory writes occur is unspecified.

8. Acknowledgments

Thank you to Matthias Kretz for useful offline discussions regarding the behaviour and contents of the permutation API.

9. Polls

9.1. Varna Meeting - 16th May 2023

POLL: SIMD permute should not support negative indices

SF	F	N	A	SA
2	10	0	0	0

POLL: SIMD permute should always pass both an index and a size to the invocable.

SF	F	N	A	SA
2	2	6	3	1

POLL: Rename `gather` to `gather_from` and `scatter` to `scatter_to` and make them member functions that do conversion and take `simd_flags<...>` (like `copy_to`/`copy_from`).

SF	F	N	A	SA
6	8	1	0	0

§ 9.2. Telecon Meeting - 2nd May 2023

Poll: We should promise more committee time to pursuing P2664R2 (Proposal to extend `std::simd` with permutation API), knowing that our time is scarce and this will leave less time for other work.

SF	F	N	A	SA
6	4	1	0	0

Poll: Pursue extended indices in the permute callable (e.g., negative indices, special values for zero element, etc.).

SF	F	N	A	SA
5	4	1	0	0

Poll: Pursue multi-index subscripting for dynamic permutations in the initial revision.

SF	F	N	A	SA
0	1	8	2	0

Poll : Pursue `operator[](simd)` subscripting for dynamic permutations in the initial revision.

SF	F	N	A	SA
1	6	2	0	0

Poll: Pursue `operator[](simd_mask)` subscripting for dynamic expand (`output[] = input`) in the initial revision.

SF	F	N	A	SA
0	0	7	2	0

§ References

§ Informative References

[DNMSO]

Matthias Kretz. *Non-Member Subscript Operator*. URL: https://web-docs.gsi.de/~mkretz/DNMSO.pdf

[Highway]

Google. *Google Highway*. URL: https://github.com/google/highway/

[P0918R1]

Tim Shen. *More simd<> Operations*. 29 March 2018. URL: https://wg21.link/p0918r1

[P1915R0]

Matthias Kretz. *Expected Feedback from simd in the Parallelism TS 2*. 7 October 2019. URL: https://wg21.link/p1915r0

[P1928R4]

Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 19 May 2023. URL: https://wg21.link/p1928r4

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: https://wg21.link/p2638r0